DS314: GENERATIVE AI & LLM SYSTEMS · ACADEMIC LAB MODULE

DS314 - Topic 1: Course Orientation & Introduction to LLM Applications

Format: Jupyter Notebook (.ipynb) **Source:** DS314_Week1_Lecture (1).ipynb **Render Engine:** KaTeX + MathML (Zero-Delim Math) **Academic Level:** 3rd Year BS Data Science

DS314 — DS Elective 1 | Generative AI

Week 1 - Course Orientation and Introduction to LLM Applications

Intended Learning Outcome (ILO): Explain course expectations and distinguish ChatGPT, general-purpose LLMs, and LLM applications within the broader generative-AI ecosystem.

Lecture

0. Scope Note: What This Course Means by "Generative AI"

0.1 AI vs. Automation

People often use "AI" to describe anything that runs without a human doing it manually. That's not quite right, and the difference matters.

- **Automation** follows fixed, pre-written rules. It doesn't "understand" anything, it just executes the same steps every time, exactly as programmed. If the input changes in a way the programmer didn't anticipate, automation breaks or does the wrong thing.
  - *Real-life example:* A vending machine is automation. Insert the right coins, press B4, get your chips. It has no idea what a "snack" is, it's just running a fixed script.
- **Artificial Intelligence (AI)** is a system that can handle input it wasn't explicitly told how to handle, by learning patterns instead of following hardcoded steps. It can generalize to new, unseen situations.
  - *Real-life example:* A spam filter that flags a phishing email it has never seen before, because it learned the *pattern* of what phishing looks like, is AI. A filter that only blocks emails from a manually maintained blacklist is automation.

The rule of thumb: if a system can only do exactly what it was told, step by step, it's automation. If a system can learn from examples and handle situations nobody explicitly programmed for, it's AI. A lot of "AI-powered" marketing on real products is actually just automation wearing an AI label, part of what this course will train you to tell apart.

0.2 AI vs. Generative AI

AI itself is a broad field. Most AI systems in daily use are not "generative" at all, they're **discriminative**: they look at input and make a decision, classification, or prediction about it. They don't create new content.

	Discriminative AI (not generative)	**Generative AI**
What it does	Classifies, predicts, or scores existing input	Creates brand-new content
Real-life example	A spam filter deciding "spam" or "not spam"	An LLM writing the reply email for you
Real-life example	A bank's fraud-detection system flagging a suspicious transaction	An LLM generating a summary of your bank statement
Real-life example	Netflix predicting what rating you'd give a movie	An LLM writing a movie recommendation blurb in your favorite genre

Generative AI is the subset of AI whose *job* is to produce new output, text, images, audio, code, that didn't exist before you asked for it. This course is entirely about the generative side, and more specifically, only the text/language side of it.

0.3 Generative AI Isn't Just LLMs

Even within "Generative AI," there are several different model families:

- **GANs** (Generative Adversarial Networks) — mostly used for image generation (e.g. deepfakes, style transfer)
- **Diffusion models** — image/video/audio generation (e.g. Stable Diffusion, Midjourney)
- **Variational Autoencoders (VAEs)** — compression and generation of structured data
- **Large Language Models (LLMs)** — text generation, **the focus of this course**

This course is scoped specifically to LLMs and NLP (Natural Language Processing). When this syllabus says "Generative AI," it means text-based generative systems, models that read and produce language, not image-generating GANs or diffusion models. If you're interested in GANs or diffusion models specifically, that's a valuable but separate area of study outside this course's scope.

0.4 What the Industry Actually Wants You to Know About LLMs

It's tempting to assume "learning AI" means learning to build a model from scratch, math, architecture, training loops. **That is not what most LLM-related jobs actually require.** Very few companies train their own foundation model, it costs millions of dollars and huge amounts of data. What companies actually hire for is the ability to:

1. **Use an existing model effectively** — writing prompts that reliably get useful output (Weeks 4-7)
2. **Connect a model to real data** — retrieval-augmented generation, so the model can answer questions about a company's own documents instead of just general knowledge (Weeks 10-13)
3. **Let a model take real actions** — function calling and agent loops, so the model doesn't just talk, it does things (booking, searching, calculating) (Week 8)
4. **Evaluate whether it's actually working** — measuring accuracy with real metrics instead of "it seemed fine" (Week 13)
5. **Know its limits and risks** — hallucination, safety, cost, when *not* to trust the output (throughout the course)

This is the actual shape of this entire syllabus. If you remember nothing else from Week 1, remember this: **you are being trained to use and integrate LLMs professionally, not to build one from scratch.** That is the real, current, in-demand skill set.

1. A Taxonomy: AI, ML, Generative AI, Models, and Applications

Terms like "AI," "model," "LLM," and "AI application" get used interchangeably in everyday conversation, but they refer to different, nested layers. Getting this right now will save you confusion for the rest of the semester.

```
Artificial Intelligence (AI)
   └── Machine Learning (ML)
         └── Deep Learning
               └── Generative AI
                     ├── GANs (images)
                     ├── Diffusion models (images/audio/video)
                     └── Large Language Models (LLMs)  ← this course
```

Term	What it actually means	Example
Artificial Intelligence (AI)	The broadest field: any system designed to perform tasks that normally require human intelligence. Includes non-learning rule-based systems too.	A rule-based tax calculator that mimics human decision-making
Machine Learning (ML)	A subset of AI where the system learns patterns from data instead of following hardcoded rules.	A spam filter trained on labeled emails
Deep Learning	A subset of ML using multi-layered neural networks.	Image recognition using a convolutional neural network
Generative AI	A subset of deep learning where the system's job is to *create* new content rather than just classify or predict.	An LLM writing an essay; a GAN generating a face
Large Language Model (LLM)	A specific type of generative AI model, trained on huge amounts of text to predict and generate language. This is a **model**, a mathematical system with trained weights.	GPT-4, Gemini, Claude, Llama
LLM Application	Software built *around* an LLM: the interface, prompts, memory, and logic that turn the raw model into something usable. This is **not** the model itself.	ChatGPT (the product) is an LLM application built around OpenAI's GPT models

The distinction that trips people up most often: a **model** is the trained mathematical system (e.g. "GPT-4" or "Gemini 2.5"). An **application** is the product built around a model (e.g. "ChatGPT" or "Google Gemini the app"). You interact with applications. Applications call models. This course teaches you to build and evaluate the *application* layer, using existing, pre-trained models, not to train the models themselves.

1.1 Going One Level Deeper: Company, Model, and Application Aren't the Same Thing

The table above still isn't specific enough to answer a question students run into constantly: **"is OpenAI the same as ChatGPT? Is ChatGPT the same as GPT? Is Cursor the same kind of thing as ChatGPT?"** The answer to all three is no, and the reason is that "application" actually splits into two different relationships with the model underneath it. There are really **four layers**, not two:

Layer	What it is	Examples
1. AI Lab / Company	The organization that trains and owns the model	OpenAI · Anthropic · Google
2. Model	The specific trained system itself, a name and version, not something you "open" directly	GPT-5, GPT-4o (by OpenAI) · Claude Sonnet 4.5, Claude Opus (by Anthropic) · Gemini 2.5 (by Google)
3. First-party application	The official product built by the **same** company, wrapped around **their own** model	ChatGPT (OpenAI, wraps GPT) · Claude.ai (Anthropic, wraps Claude) · Gemini app (Google, wraps Gemini)
4. Third-party application	A product built by a **different** company, using another company's model through an API	Cursor, Windsurf (independent companies; can call OpenAI, Anthropic, or Google models depending on settings) · GitHub Copilot (Microsoft, uses OpenAI models) · Perplexity (independent, uses multiple models)

Worked examples, to make this concrete:

- **OpenAI vs. ChatGPT vs. GPT** — OpenAI is the *company* (layer 1). GPT (e.g. GPT-5) is the *model* (layer 2), the trained system itself. ChatGPT is the *first-party application* (layer 3), the product OpenAI built around their own model.
- **Anthropic vs. Claude.ai vs. Sonnet** — Anthropic is the *company* (layer 1). Claude Sonnet is the *model* (layer 2). Claude.ai is the *first-party application* (layer 3). Note: "Claude" alone is often used loosely to mean both the model family and the app, that ambiguity is common even among professionals, so pay attention to context.
- **Cursor and Windsurf** — these are *third-party applications* (layer 4). Unlike ChatGPT, they are not tied to one company's model, they're built by independent companies that let you choose which model (OpenAI's, Anthropic's, Google's) actually powers the tool underneath. This is a fundamentally different relationship than ChatGPT has to GPT.
- **GitHub Copilot** — also a *third-party application* (layer 4): built by Microsoft/GitHub, but historically powered by OpenAI's models via API, a different company's application wrapped around a different company's model.

Why this distinction matters practically: first-party applications (ChatGPT, Claude.ai, Gemini) usually give you the most direct, full-featured access to that company's latest model. Third-party applications (Cursor, Copilot, Perplexity) add their own extra features and interface on top, but you're also depending on two companies instead of one, the model provider and the application builder, which affects things like pricing, rate limits, and which specific model version is actually running underneath at any given time.

2. What Is an LLM, and Why Is It Called That?

LLM stands for Large Language Model. Breaking down the name piece by piece actually explains what it is:

- **Language** — it works with human language (text): reading it, understanding patterns in it, and producing more of it. Not images, not sound, unless a specific version is additionally trained for that, language is the core.
- **Model** — in the ML sense, a "model" is a mathematical system trained on data to make predictions. Here, specifically, an LLM is trained to do one very simple-sounding thing: **predict the next word (token) in a sentence, given everything before it.** That's it. That one simple training task, repeated across an enormous amount of text, is what produces something that can hold a conversation, write code, or summarize an article.
- **Large** — "large" refers to two things at once: the huge amount of text the model was trained on (often a large fraction of the public internet, books, and articles), and the huge number of internal parameters (the model's internal "settings," sometimes in the billions) it uses to make its predictions. Older language models existed with far fewer parameters and far less data, "large" is what separates today's LLMs from their much smaller ancestors.

A simple analogy: imagine a student who has read an enormous number of books, articles, and conversations, so many that they've absorbed the *patterns* of how language works, how sentences flow, how ideas connect, without memorizing everything word for word. When you give them the start of a sentence, they can make a very well-informed guess about what comes next. Do that one word at a time, over and over, and you get a full, coherent response. That's essentially what an LLM does, just at a scale and speed no human could match.

2.1 A Short History: Where LLMs Came From

LLMs didn't appear overnight. They're the result of decades of incremental progress:

- **Early language models (pre-2010s)** — simple statistical models that predicted the next word based on the last one or two words only (called n-gram models). Useful for basic autocomplete, but with no real understanding of longer context.
- **Neural language models (2010s)** — language models built with neural networks (RNNs, LSTMs) that could remember a bit more context than pure statistics, still limited, and slow to train on long text.
- **The Transformer architecture (2017)** — a research paper titled "*Attention Is All You Need*" introduced the Transformer, a new architecture that could look at an entire sentence (or document) at once using a mechanism called **self-attention**, instead of reading word by word. This single breakthrough is the direct ancestor of every major LLM you've heard of.
- **Scaling up (2018-2020)** — researchers found that if you took the Transformer architecture and simply made it much *bigger* (more parameters, more training text), it kept getting better in ways that weren't fully predictable in advance. This is where "large" language models, in the modern sense, actually begin. OpenAI's GPT series and Google's BERT are early, well-known examples from this era.
- **The chat-application era (2022-present)** — models got wrapped in conversational, instruction-following interfaces, most visibly ChatGPT's public release in late 2022, which is what took LLMs from "a research topic" to "something your relatives use." This is also, not coincidentally, the point where LLM *applications* (Section 3, next) became a massive category of their own, separate from the models underneath them.

Why this history matters for you: every skill in this course, prompting, RAG, agents, evaluation, exists specifically *because* of the Transformer breakthrough and the scaling era that followed it. Understanding that LLMs are a fairly recent, rapidly-evolving technology, not a finished, static field, is part of why continued learning and staying current matters so much in this space (you'll see this again as Program Outcome **I** later in the syllabus).

3. LLM Applications as a Key Interface to the New AI

Think of the relationship between a **model** and an **application** the way you'd think about a car engine and the car itself. The engine (the model) is powerful, but on its own it's just a block of metal on a table, you can't drive it anywhere. The car (the application) is what wraps around the engine: the steering wheel, the seats, the dashboard, so an ordinary person can actually use that power to go somewhere useful.

Real-life examples of this layering:

- **Notion AI** is an LLM application built on top of models like GPT and Claude, wrapped around Notion's note-taking interface. You never see "the model," you just see "summarize this page" and "write me an outline" buttons.
- **GitHub Copilot** is an LLM application built on top of OpenAI's Codex/GPT models, wrapped directly inside your code editor. You type a comment, it suggests the code.
- **Duolingo Max** is an LLM application built on top of GPT models, wrapped around Duolingo's language-learning exercises, "Explain My Answer" and roleplay conversations.

In every case above, the underlying model is doing the heavy lifting, but you never interact with the model directly. You interact with the application built around it. This is why LLM applications are called the "key interface" to modern AI: for the overwhelming majority of people, an LLM application *is* their entire experience of what "AI" is.

4. LLM Applications and the Universalization of AI

Before LLM applications became mainstream, using AI meant writing code, training models, or working with structured data pipelines, a task reserved for people with a technical background. LLM applications changed that: anyone who can type a sentence can now direct an AI system to do useful work.

A real-life comparison: imagine a small sari-sari store owner who wants to write a Facebook ad for a promo. Ten years ago, she'd either write it herself with no marketing background, or pay someone to do it. Today, she can open ChatGPT, type "write me a catchy Facebook ad for a 3-for-100 chips promo," and get a usable draft in seconds, no coding, no hiring, no technical skill required. That's universalization: AI capability becoming accessible to non-technical, everyday users through natural language, not just to engineers through code.

This shift is also why this course's own philosophy matters: most students will not be training or fine-tuning models professionally. Far more of you will be **using, integrating, and evaluating** LLM applications, which is exactly the skill path this syllabus is built around, and exactly why section 0.4 above matters more than it might have seemed at first.

5. ChatGPT versus Purpose-Built LLM Applications

It's easy to think "ChatGPT" and "LLM application" mean the same thing. They don't.

	General-purpose (e.g. ChatGPT, Gemini, Claude)	**Purpose-built (e.g. GitHub Copilot, Jasper, Duolingo Max)**
Scope	Broad, can attempt almost any text-based task	Narrow, tuned for one specific job
Interface	Open chat window	Often embedded inside another tool (an IDE, a language app, a CRM)
Customization	Minimal, general system prompt	Heavily customized system prompt, tools, and sometimes retrieval over specific data
Autonomy	Usually low, answers when asked	Can range from low to high depending on the product

Real-life comparison: if you open ChatGPT and ask it to help you learn Spanish, it can, in a general, open-ended way. But Duolingo Max is *specifically* built for language learning: it tracks your progress, adjusts difficulty, and gives feedback tuned to a curriculum. Under the hood, both might use a similar type of model. What's different is the application layer built around it, one is a generalist, the other is a specialist.

Another real-life comparison: a customer support chatbot on Globe or Smart's website is purpose-built, it's tuned specifically to answer billing and plan questions, often connected to your actual account data. It would do a *worse* job than ChatGPT at writing you a poem, but a much *better* job than ChatGPT at telling you your exact current bill, because it's connected to real, private company data that ChatGPT has no access to.

Both are "LLM applications." The difference is scope, customization, and what data they're connected to, not the underlying technology. You'll see this distinction come back in Week 3, when we classify applications by **autonomy level**.

6. A First Look at the Generative-AI Application Landscape

To give you a working map before we go further, here are common categories of LLM applications you'll encounter, with a real example and what makes each one distinct:

- **Conversational assistants** — ChatGPT, Gemini, Claude. General-purpose, open-ended, answer almost anything.
- **Coding assistants** — GitHub Copilot, Cursor. Purpose-built for writing and explaining code, embedded directly inside a code editor.
- **Writing/marketing assistants** — Jasper, Copy.ai. Purpose-built for ad copy, blog posts, and marketing content, often with brand-voice customization.
- **Search/research assistants** — Perplexity. Purpose-built to answer questions with cited, up-to-date sources, rather than relying purely on what the model memorized during training.
- **Customer-facing bots** — support chatbots embedded in a company's website, connected to that company's real account/billing data.
- **Agentic tools** — systems that can take multi-step actions on your behalf, like booking a flight or filling out a form autonomously. We'll build one ourselves in Week 8.

You are not expected to memorize this list, it's here to give you a mental map for the laboratory activity below.

Laboratory

Practical Workshop: LLM Application Landscape

Goal: Explore real LLM applications and categorize them, so you build an accurate mental model of the ecosystem before we start building inside it from Week 4 onward.

Instructions:

1. Visit and briefly try **at least four** different LLM applications. Suggestions:
   - A general-purpose assistant (ChatGPT, Gemini, Claude)
   - A coding assistant (GitHub Copilot, if available to you, or read its product page)
   - A writing/marketing tool (Jasper, Copy.ai)
   - A search/research assistant (Perplexity)
   - Any customer-support chatbot you can find on a company website
2. For each one, fill in the table in the code cell below (edit the Python dictionary directly, no API calls needed this week).
3. Answer the reflection questions at the end.

In [1]:

```
# Fill in this dictionary as you explore each application.
# scope: "general-purpose" or "purpose-built"
# autonomy: "low" (just answers), "medium", or "high" (can take multi-step actions)

app_landscape = {
    "ChatGPT": {
        "scope": "",          # e.g. "general-purpose"
        "primary_use": "",    # e.g. "conversational assistant"
        "autonomy": "",       # e.g. "low"
        "notes": ""
    },
    "Gemini": {
        "scope": "",
        "primary_use": "",
        "autonomy": "",
        "notes": ""
    },
    "GitHub Copilot": {
        "scope": "",
        "primary_use": "",
        "autonomy": "",
        "notes": ""
    },
    "Perplexity": {
        "scope": "",
        "primary_use": "",
        "autonomy": "",
        "notes": ""
    },
    # Add a 5th app of your own choice below
    "": {
        "scope": "",
        "primary_use": "",
        "autonomy": "",
        "notes": ""
    },
}

# Run this cell to display your completed table
import pandas as pd
pd.DataFrame(app_landscape).T
```

Out [1]:

	scope	primary_use	autonomy	notes
ChatGPT				
Gemini				
GitHub Copilot				
Perplexity				

Reflection Questions

Answer briefly in the markdown cell below (double-click to edit):

1. Which application felt the most "general-purpose," and which felt the most "purpose-built"? What specifically made you categorize them that way?
2. Did any application surprise you in terms of how much autonomy it seemed to have?
3. Based on today's lecture, where do you think most real industry jobs in this field actually focus: building brand-new models, or building/integrating applications on top of existing ones? Why?

In [ ]:

```
# Your reflection answers here (as a Python multi-line string is fine, or edit the markdown cell above

reflection = """
1.
2.
3.
"""
print(reflection)
```